



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #4

**freeRTOS: TASKS, SYNCHRONIZATION &
COMMUNICATION WITH MAILBOXES**

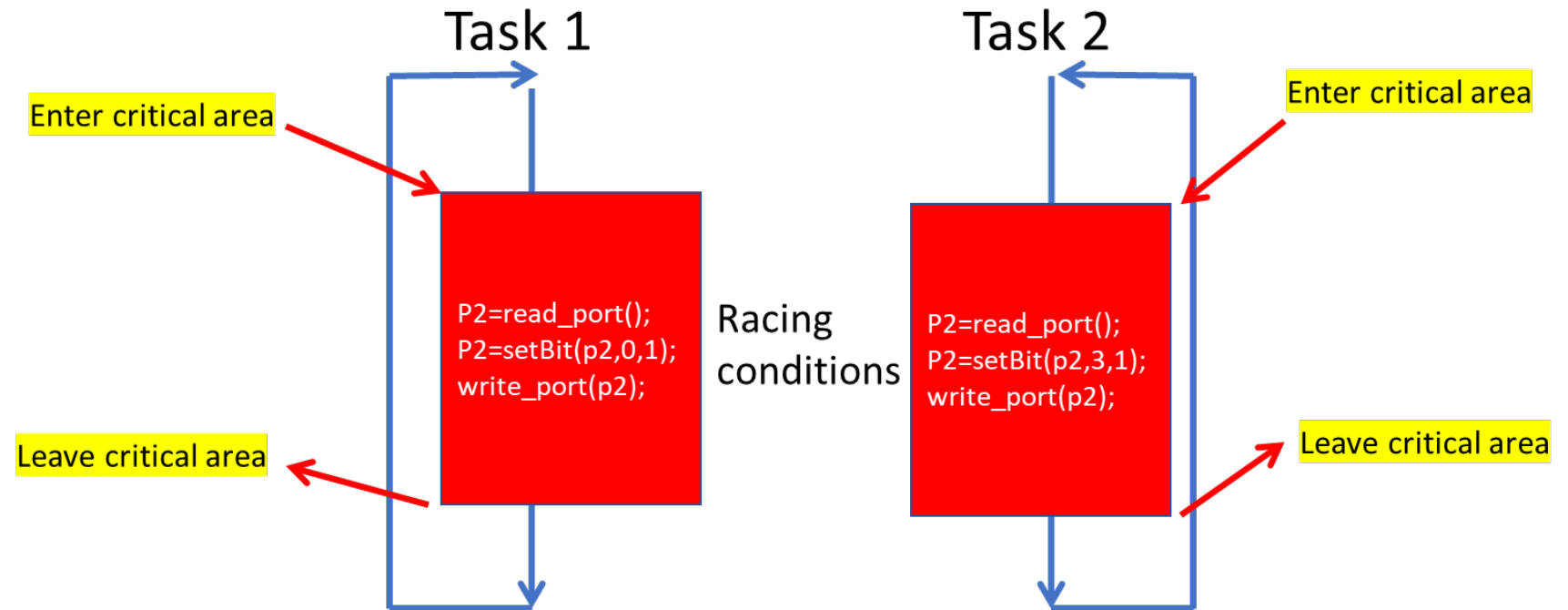
2022 - 2023

freeRTOS: Tasks, synchronization & communication with mailboxes

- ☐ Race condition review
- ☐ Semaphores review
- ☐ Mailboxes
- ☐ Labwork 1: First tasks identification and interactions (synchronization & communication)

Race Condition: Storage Kit (access to Port 2) [REVIEW]

- ❑ Task 1 and Task 2 are running a fragment of code, which cannot be executed simultaneously
- ❑ We need to specify **critical areas / sections**.
- ❑ A **critical area** is a block of code which while running, everything else in the system freezes until the code inside the block finishes and execution exits from that area.



Race Condition: Storage Kit (access to Port 2) [REVIEW]

- ❑ `taskENTER_CRITICAL()` and `taskEXIT_CRITICAL()` operate at kernel level, meaning when inside a critical area, “everything” else inside the kernel stops working until exiting the critical area.
- ❑ As such, the code inside critical areas should work as fast as possible (no loops, no sleeps,... no anything causing latency).
- ❑ For less strident conditions, the recommended alternative is to model mutual exclusion with semaphores.

```
void vTaskVertical(void* pvParameters)
{
    while (TRUE)
    {
        //go up
        taskENTER_CRITICAL();
        uint8 aa = readDigitalU8(2);
        writeDigitalU8(2, (aa & (0xff - 0x04)) | 0x08);
        taskEXIT_CRITICAL();

        // wait till last sensor
        while ((readDigitalU8(0) & 0x40)) { taskYIELD(); }

        // go left
        taskENTER_CRITICAL();
        aa = readDigitalU8(2);
        vTaskDelay(10); // simulate some latency
        writeDigitalU8(2, (aa & (0xff - 0x08)) | 0x04);
        taskEXIT_CRITICAL();

        // wait till last sensor
        while ((readDigitalU8(1) & 0x08)) { taskYIELD(); }
    }
}
```

Example of a CRITICAL AREA - Storage Kit (access to Port 2)



- ❑ If both task_1 and task_2 run slightly at the same time, they might read the same value from port 2.
- ❑ In this situation, the value for port 2 may overlap.
- ❑ As such, either the system will run axis x right, axis z up, or both. But we don't know which.
- ❑ We need to ensure that one task works without undermining other tasks.

```
void task_1(void *)
{
    move_x_right();
}

void move_x_right()
{
    uInt8 p;
    p = readDigitalU8 (2);
    setBitValue(&p,7,1);
    setBitValue(&p,6,0);
    writeDigitalU8 (2,p);
}
```

```
void task_2(void *)
{
    move_z_up();
}

void move_z_up()
{
    uInt8 p;
    p = readDigitalU8 (2);
    setBitValue(&p,3,1);
    setBitValue(&p,2,0);
    writeDigitalU8 (2,p);
}
```

Example of a CRITICAL AREA - Storage Kit (access to Port 2)



Solution: Set code as **critical**, so that it can run in exclusive mode. This means no other tasks or interrupts can run before exiting the critical code.

But be careful: if loops or long operations are created inside **critical** areas, the whole program, including the kernel, will freeze until exiting that critical area (see next slide).



```
void move_x_right()
{
    uInt8 p;
    taskENTER_CRITICAL();
    p = readDigitalU8 (2);
    setBitValue(&p,7,1);
    setBitValue(&p,6,0);
    writeDigitalU8 (2,p);
    taskEXIT_CRITICAL();
}
```

```
void move_z_up()
{
    uInt8 p;
    taskENTER_CRITICAL();
    p = readDigitalU8 (2);
    setBitValue(&p,3,1);
    setBitValue(&p,2,0);
    writeDigitalU8 (2,p);
    taskEXIT_CRITICAL();
}
```



```
void some_silly_function()
{
    taskENTER_CRITICAL();
    while(some_lasting_condition) {
        do_something_taking_long_time();
    }
    taskEXIT_CRITICAL();
}
```

All the other TASKS and interrupts will freeze during the while loop, basically, “killing” the system.

Therefore, critical areas should contain very fast code ONLY !!!!

[KERNEL](#)[LIBRARIES](#)[SUPPORT](#)[PARTNERS](#)[COMMUNITY](#)[Download FreeRTOS](#)[Home](#)[Getting Started](#)[FreeRTOS Books](#)[About FreeRTOS Kernel](#)[Developer Docs](#)[Secondary Docs](#)[Supported Devices](#)[API Reference](#)[Task Creation](#)[Task Control](#)[Task Utilities](#)[RTOS Kernel Control](#)[taskYIELD\(\)](#)[taskENTER_CRITICAL\(\)](#)[taskEXIT_CRITICAL\(\)](#)[taskENTER_CRITICAL_FROM_ISR\(\)](#)[taskEXIT_CRITICAL_FROM_ISR\(\)](#)[taskDISABLE_INTERRUPTS\(\)](#)[taskENABLE_INTERRUPTS\(\)](#)[vTaskStartScheduler\(\)](#)[vTaskEndScheduler\(\)](#)[vTaskSuspendAll\(\)](#)[xTaskResumeAll\(\)](#)[vTaskStepTick\(\)](#)[xTaskCatchUpTicks\(\)](#)

Kernel Control

[\[API\]](#)

Modules

- [taskYIELD](#)
- [taskENTER_CRITICAL](#)
- [taskEXIT_CRITICAL](#)
- [taskENTER_CRITICAL_FROM_ISR](#)
- [taskEXIT_CRITICAL_FROM_ISR](#)
- [taskDISABLE_INTERRUPTS](#)
- [taskENABLE_INTERRUPTS](#)
- [vTaskStartScheduler](#)
- [vTaskEndScheduler](#)
- [vTaskSuspendAll](#)
- [xTaskResumeAll](#)
- [vTaskStepTick](#)

It is highly recommended that you explore all the possible kernel control functionalities!

Detailed Description

taskYIELD

task. h

`taskYIELD()` is used to request a context switch to another task. However, if there are no other tasks at a higher or equal priority to the task that calls `taskYIELD()` then the RTOS scheduler will simply select the task that called `taskYIELD()` to run again.

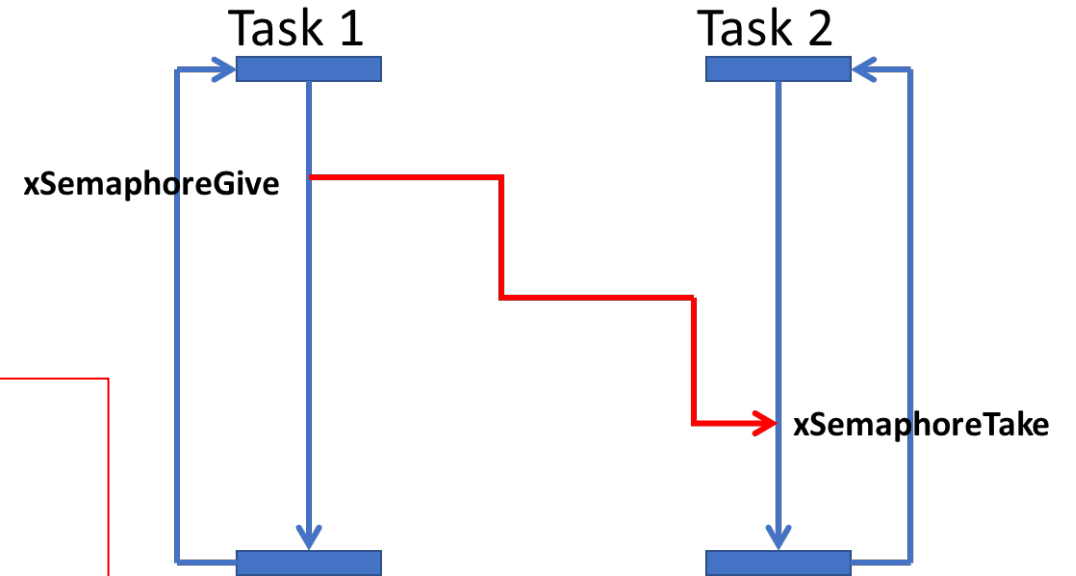
TASK SYNCHRONIZATION (SEMAPHORES REVIEW)

```
// declare the semaphore on top of the file, just below #defines  
xSemaphoreHandle xSemaphore = NULL;
```

```
void vTask_signals(void* pvParameters)  
{  
    printf("write something\n");  
    while (1) {  
        char ch[101];  
        gets_s(ch, 100); // input from keyboard  
        printf("\n vTask_signals got string '%s' from keyboard...", ch);  
        if (strcmp(ch, "fim") == 0)  
            exit(0); // terminate program...  
        xSemaphoreGive(xSemaphore); //V  
    }  
}
```

```
void vTask_waits(void* pvParameters)  
{  
    while (1) {  
        if (xSemaphoreTake(xSemaphore, 10000) == pdTRUE) //P  
            printf("\nvTask_waits has been awoken\n");  
        else  
            printf("\nvTask_waits is tired of waiting\n");  
    }  
}
```

```
void myDaemonTaskStartupHook(void) {  
    xSemaphore = xSemaphoreCreateCounting(10, 0);  
    xTaskCreate(vTask_waits, "vTask_waits", 100, NULL, 0, NULL);  
    xTaskCreate(vTask_signals, "vTaskCode2", 100, NULL, 0, NULL);  
}
```



[KERNEL](#)[LIBRARIES](#)[SUPPORT](#)[PARTNERS](#)[COMMUNITY](#)[Download FreeRTOS](#)[Home](#)[Getting Started](#)[FreeRTOS Books](#)[About FreeRTOS Kernel](#)[Developer Docs](#)[Secondary Docs](#)[Supported Devices](#)[API Reference](#)[Task Creation](#)[Task Control](#)[Task Utilities](#)[RTOS Kernel Control](#)[Direct To Task Notifications](#)[Queues](#)[Queue Sets](#)[Stream Buffers](#)[Message Buffers](#)[Semaphore / Mutexes](#)[xSemaphoreCreateBinary\(\)](#)[xSemaphoreCreateBinaryStatic\(\)](#)[vSemaphoreCreateBinary\(\)](#)[xSemaphoreCreateCounting\(\)](#)[xSemaphoreCreateCountingStatic\(\)](#)[xSemaphoreCreateMutex\(\)](#)[xSemaphoreCreateMutexStatic\(\)](#)[xSem'CreateRecursiveMutex\(\)](#)[xSem'CreateRecursiveMutexStatic\(\)](#)

Semaphores

[\[API\]](#)

TIP: 'Task Notifications' can provide a light weight alternative to semaphores in many situations

Modules

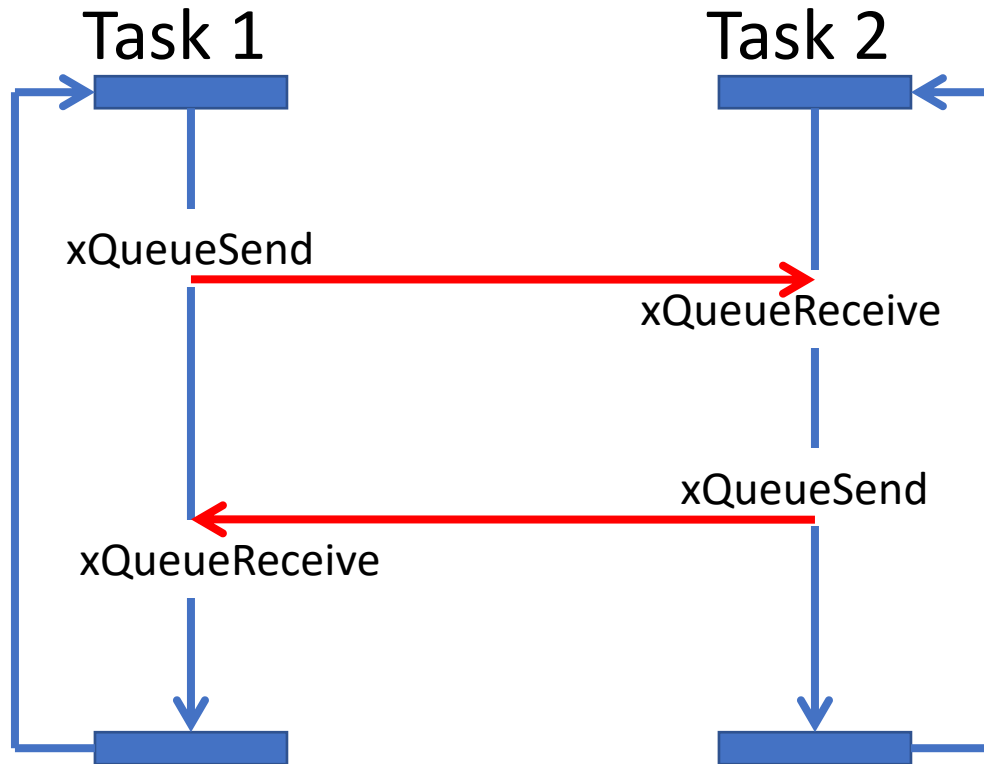
- [xSemaphoreCreateBinary](#)
- [xSemaphoreCreateBinaryStatic](#)
- [vSemaphoreCreateBinary](#) [use [xSemaphoreCreateBinary\(\)](#) for new designs]
- [xSemaphoreCreateCounting](#)
- [xSemaphoreCreateCountingStatic](#)
- [xSemaphoreCreateMutex](#)
- [xSemaphoreCreateMutexStatic](#)
- [xSemaphoreCreateRecursiveMutex](#)
- [xSemaphoreCreateRecursiveMutexStatic](#)
- [vSemaphoreDelete](#)
- [xSemaphoreGetMutexHolder](#)
- [xSemaphoreTake](#)
- [xSemaphoreTakeFromISR](#)
- [xSemaphoreTakeRecursive](#)
- [xSemaphoreGive](#)
- [xSemaphoreGiveRecursive](#)
- [xSemaphoreGiveFromISR](#)
- [uxSemaphoreGetCount](#)

vSemaphoreDelete

[semphr.h](#)

It is highly recommended that you explore all semaphore typology and functionalities!

TASK COMMUNICATION (MAILBOXES / QUEUES)



- ❑ Two parallel tasks sending/ receiving data through **MAILBOXES**
- ❑ One task sends a message to another and vice-versa

TASK COMMUNICATION (MAILBOXES / QUEUES)

□ Let's update project `demo_rtos.sln` (in C:\str\demo_rtos) with mailboxes

```
// declare the mailbox on top of the file, just below #defines
xQueueHandle xQueue = NULL;
```

```
void vTaskCode_MbxSender (void * pvParameters){
    double val = 0;
    for (;;) {
        // send real numbers to another task
        // sender is faster than receiver
        val = val + 1;
        xQueueSend(xQueue, &val, 0);
        vTaskDelay(50); //50ms
    }
}
```

```
void vTaskCode_MbxReceiver (void * pvParameters){
    double val;
    for (;;) {
        // receive real numbers from the sender
        // sender is faster than receiver
        xQueueReceive(xQueue, &val, portMAX_DELAY);
        printf("\nreceived_val =%1.3f", val);
        vTaskDelay(1000); //1s
    }
}
```

```
void myDaemonTaskStartupHook (void) {
    // declare and create a mailbox
    xQueue = xQueueCreate(10, sizeof(double));

    xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, NULL);
    xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, NULL);
    (...)
}
```

```
void main(){
    initialiseHeap();
    vApplicationDaemonTaskStartupHook = &myDaemonTaskStartupHook;
    vTaskStartScheduler();
}
```



What happens?

TASK COMMUNICATION (MAILBOXES / QUEUES)

C:\str\demo_rtos\Debug\demo_rtos.

```
received_val =1.000  
received_val =2.000  
received_val =3.000  
received_val =4.000  
received_val =5.000  
received_val =6.000  
received_val =7.000  
received_val =8.000  
received_val =9.000  
received_val =10.000  
received_val =11.000  
received_val =21.000  
received_val =41.000  
received_val =61.000  
received_val =82.000  
received_val =103.000  
received_val =124.000  
received_val =144.000 .  
received_val =183.000  
received_val =203.000 .  
received_val =223.000  
received_val =243.000  
received_val =263.000 .  
received_val =283.000  
received_val =303.000 .  
received_val =323.000  
received_val =343.000
```



- Now try to change the `vTaskDelay` to 1s in `vTaskCode_MbxSender`:

```
void vTaskCode_MbxSender (void * pvParameters){  
    xQueueHandle xQueue = (xQueueHandle)pvParameters;  
    double val = 0;  
    for (;;) {  
        // send real numbers to another task  
        // sender is faster than receiver  
        val = val + 1;  
        xQueueSend(xQueue, &val, 0);  
        vTaskDelay(1000);  
    }  
}
```



And now?
What happens?

TASK COMMUNICATION (MAILBOXES / QUEUES)

```
C:\str\demo_rtos\De...
received_val =1.000
received_val =2.000
received_val =3.000
received_val =4.000
received_val =5.000
received_val =6.000
received_val =7.000
received_val =8.000
received_val =9.000
received_val =10.000
received_val =11.000
received_val =12.000
received_val =13.000
received_val =14.000
received_val =15.000
received_val =16.000
received_val =17.000
received_val =18.000
received_val =19.000
received_val =20.000
received_val =21.000
received_val =22.000
received_val =23.000
received_val =24.000
received_val =25.000
received_val =26.000
received_val =27.000
received_val =28.000_
```



- Now try to change the *Mbx size* to 100 and keep the *vTaskDelay* to 50ms in *vTaskCode_MbxSender*:

```
void myDaemonTaskStartupHook (void) {
    // declare and create a mailbox
    xQueueHandle xQueue = xQueueCreate(100, sizeof(double));

    xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, NULL);
    xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, NULL);
    (...)
}
```



What is your
conclusion?

TASK COMMUNICATION (MAILBOXES / QUEUES)



□ Let's send a structured message with the **Part** information

```
// declare the mailbox on top of the file, just below #defines
xQueueHandle mbx_part;

typedef struct part {
    char player_name[50];
    int x_position;
    int z_position;
} Part;
```

```
void myDaemonTaskStartupHook(void) {
    mbx_part = xQueueCreate(10, sizeof(Part));
    xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, NULL, 0, NULL);
    xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, NULL, 0, NULL);
}
```

```
void vTaskCode_MbxSender(void* pvParameters){
    Part part_info;

    strcpy_s(part_info.player_name, sizeof(part_info.player_name), "Luis");
    part_info.x_position = 2;
    part_info.z_position = 1;

    xQueueSend(mbx_part, &part_info, portMAX_DELAY);
}
```

```
void vTaskCode_MbxReceiver(void* pvParameters){
    Part part_info;

    xQueueReceive(mbx_part, &part_info, portMAX_DELAY);
    printf("\n Player Name = %s\n X Position = %d\n Y Position = %d\n",
        part_info.player_name, part_info.x_position, part_info.x_position);
}
```

[KERNEL](#)[LIBRARIES](#)[SUPPORT](#)[PARTNERS](#)[COMMUNITY](#)[Download FreeRTOS](#)[Home](#)[Getting Started](#)[FreeRTOS Books](#)[About FreeRTOS Kernel](#)[Developer Docs](#)[Secondary Docs](#)[Supported Devices](#)[API Reference](#)[Task Creation](#)[Task Control](#)[Task Utilities](#)[RTOS Kernel Control](#)[Direct To Task Notifications](#)[Queues](#)[xQueueCreate\(\)](#)[xQueueCreateStatic\(\)](#)[vQueueDelete\(\)](#)[xQueueSend\(\)](#)[xQueueSendFromISR\(\)](#)[xQueueSendToBack\(\)](#)[xQueueSendToBackFromISR\(\)](#)[xQueueSendToFront\(\)](#)[xQueueSendToFrontFromISR\(\)](#)[xQueueReceive\(\)](#)[xQueueReceiveFromISR\(\)](#)[uxQueueMessagesWaiting\(\)](#)[uxQueueMessagesWaitingFromISR\(\)](#)[uxQueueSpacesAvailable\(\)](#)[xQueueReset\(\)](#)

Queue Management

[\[API\]](#)

Modules

- [xQueueCreate](#)
- [xQueueCreateStatic](#)
- [vQueueDelete](#)
- [xQueueSend](#)
- [xQueueSendFromISR](#)
- [xQueueSendToBack](#)
- [xQueueSendToBackFromISR](#)
- [xQueueSendToFront](#)
- [xQueueSendToFrontFromISR](#)
- [xQueueReceive](#)
- [xQueueReceiveFromISR](#)
- [uxQueueMessagesWaiting](#)
- [uxQueueMessagesWaitingFromISR](#)
- [uxQueueSpacesAvailable](#)
- [xQueueReset](#)
- [xQueuePeek](#)
- [xQueuePeekFromISR](#)
- [vQueueAddToRegistry](#)
- [pcQueueGetName](#)
- [vQueueUnregisterQueue](#)
- [xQueueIsQueueEmptyFromISR](#)
- [xQueueIsQueueFullFromISR](#)
- [xQueueOverwrite](#)
- [xQueueOverwriteFromISR](#)

Detailed Description

It is highly recommended that you explore all mailboxes/queues typology and functionalities!

Lab Work 1 - Automated Pharmacy Storage System

Req.	Description
Storage utilization requirements	
FR1	Using the keyboard, develop a robust (semi-automatic) storage calibration. Using keys, the user defines the direction of the movements. During a movement, the actuator must stop when it reaches a position sensor.
FR2	Store a part in the storage. The system will store the part with the current date, player, and cell(x,y). The system should verify if the cell is empty.
FR3	Remove part from cell (x,z) specified through keyboard.
FR4	When starting a new game, the user must be able to select the difficulty of the game (time available to play). Easy = 10 seconds, Hard = 5 seconds.
FR5	The system must allow three types of games: 1. Normal game: The users can only add parts. 2. Hybrid game: The users can add or remove a part from the system (the player can remove one part from the opponent). 3. Random game. When the users get their turn, the system will say if the user can add or remove a part (randomly).
FR6	Activate “flashing lamp” (at the left) to alert the player expired the time available to play and lose turn.
FR7	When the player loses turn because of time expiring (FR6), the system should automatically remove one of the parts already played by this player (randomly). The “flashing lamp” (at the left) must be activated until the part is removed.
FR8	While being operated (moving), if both buttons are pressed, then perform emergency

FR14	Given a part type (crosses and circles), shows all corresponding (x,z) coordinates.
FR15	Show the record (historical) of all parts removed from the storage.
Non-Functional requirements:	
NF1	While operating, the system can provide information (e.g., occupation level / free cells / part list).
NF2	The system works appropriately (one part per cell, not taking parts from empty cells)
NF3	The system always moves within limits x, y, z.
NF4	The system moves along (x,z) only when y is stopped at center position (y==2).
NF5	The system delivers and receives products with y out (y=3) in coordinate (x,y) = (1,1)

□ Let's architect our project solution!



➤ Need a solution to perform the “play” functionalities!

- Menu showing functionalities
- Request Player info and ask for (X,Z) intended location
- Performing storing action

❖ Which tasks?

❖ Need tasks synchronization?

❖ Need tasks communication? (information exchange)

➤ Need a solution to perform the “remove part” functionalities!

- Menu showing functionalities
- Ask for (X,Z) or part code for intended delivering
- Performing delivering action

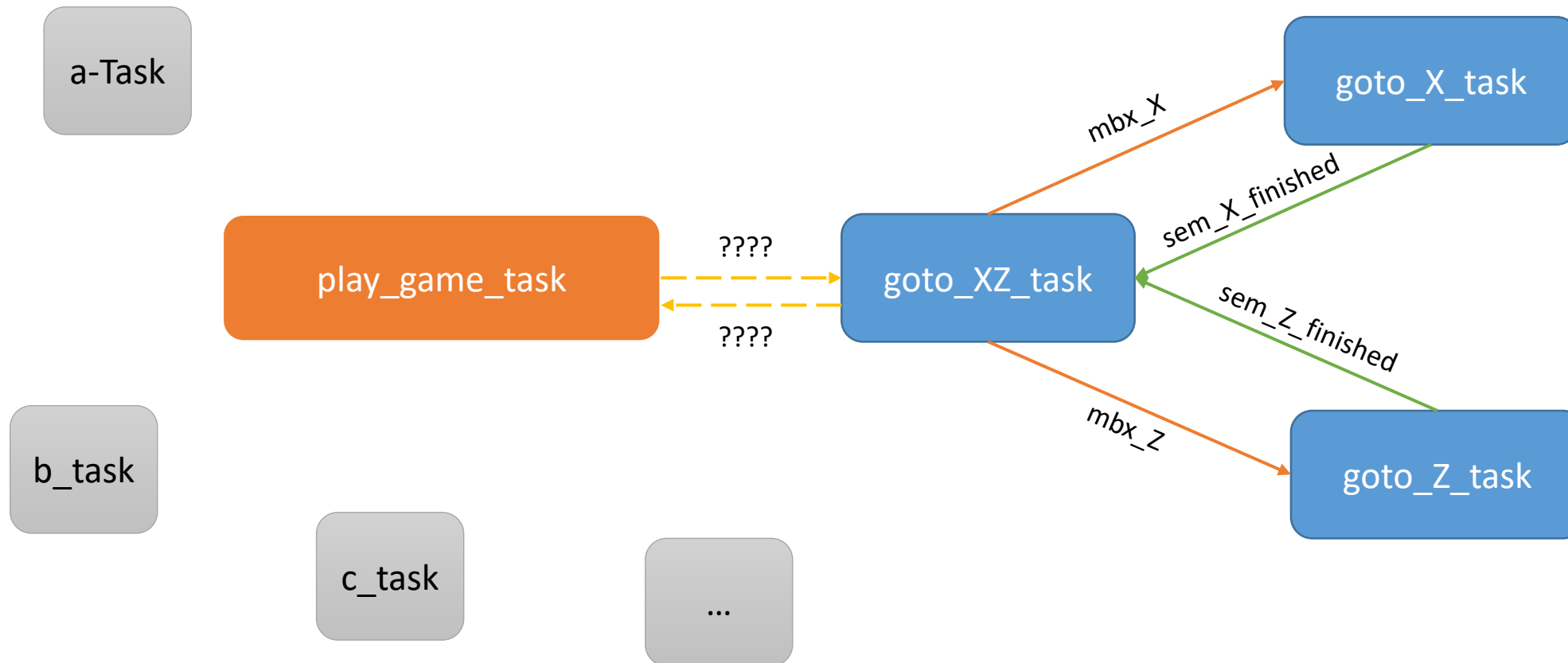
❖ Which tasks?

❖ Need tasks synchronization?

❖ Need tasks communication? (information exchange)

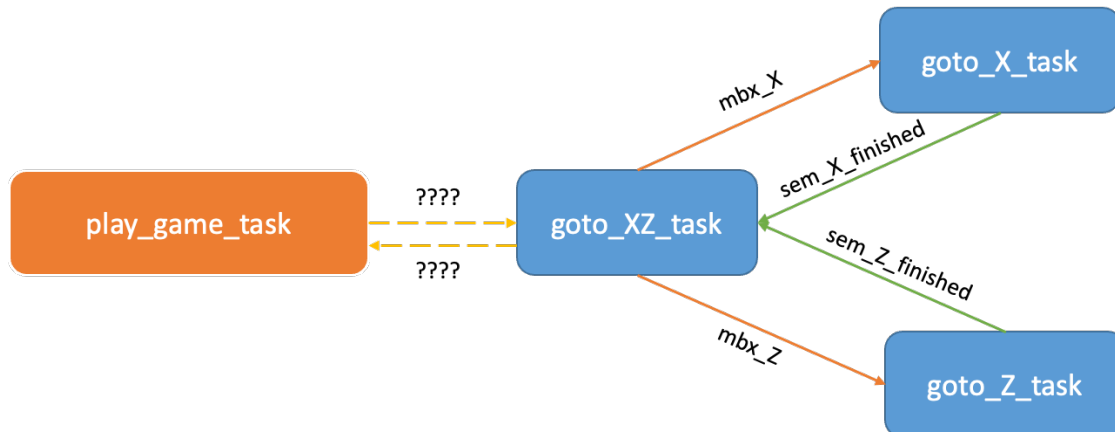
➤ Need a solution to ...

□ Let's start by designing a diagram of tasks and inter-task interactions:



 **Complete the diagram!**

- Implementing the designed tasks and interactions (the blue ones so far...)



- ✓ Creation of mailbox `mbx_X` to communicate with `goto_X_task`
- ✓ Creation of mailbox `mbx_Z` to communicate with `goto_Z_task`
- ✓ Creation of semaphores `sem_X_finished`, `sem_Z_finished`
- ✓ Creation of `goto_X_task`
- ✓ Creation of `goto_Z_task`
- ✓ Creation of `goto_XZ_task`
- ✓ Ask coordinates (X,Z) from the keyboard

□ Definition and creation of semaphores and mailboxes

```
//Semaphores and Mailboxes declaration
xSemaphoreHandle sem_X_finished;
xSemaphoreHandle sem_Z_finished;

xQueueHandle mbx_X; //for goto_X
xQueueHandle mbx_Z; //for goto_Z
```

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();

    sem_X_finished = xSemaphoreCreateCounting(10, 0);
    sem_Z_finished = xSemaphoreCreateCounting(10, 0);

    mbx_X = xQueueCreate(10, sizeof(int));
    mbx_Z = xQueueCreate(10, sizeof(int));

}
```

□ Definition and creation of goto_X_task, goto_Z_task and goto_XZ_task

```
void goto_X_task(void* pvParameters) {
    int x;
    while (TRUE) {
        xQueueReceive(mbx_X, &x, portMAX_DELAY);
        gotoX(x);
        xSemaphoreGive(sem_X_finished); // signals task completion
    }
}
```

```
void goto_Z_task(void* pvParameters) {
    int z;
    while (TRUE) {
        xQueueReceive(mbx_Z, &z, portMAX_DELAY);
        gotoZ(z);
        xSemaphoreGive(sem_Z_finished); // signals task completion
    }
}
```

send **request** for goto_X_task and goto_Z_task which execute gotoX and gotoZ on their own.

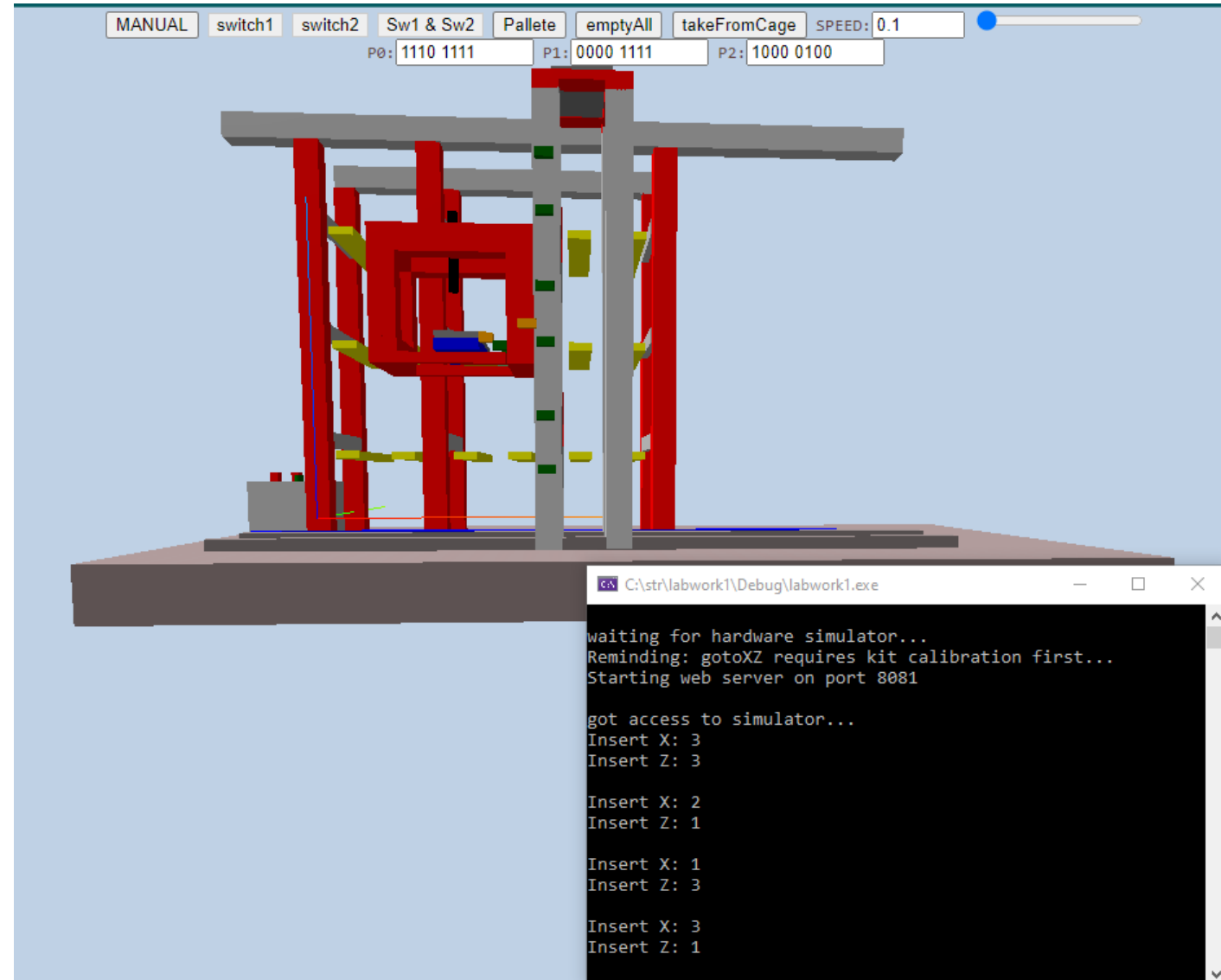
```
void goto_XZ_task(void* pvParameters) {
    int x = 0, z = 0;
    while (TRUE) {
        printf("\nInsert X: ");
        scanf_s("%d", &x);
        printf("Insert Z: ");
        scanf_s("%d", &z);
        xQueueSend(mbx_X, &x, portMAX_DELAY);
        xQueueSend(mbx_Z, &z, portMAX_DELAY);
        xQueueSemaphoreTake(sem_X_finished, portMAX_DELAY);
        xQueueSemaphoreTake(sem_Z_finished, portMAX_DELAY);
    }
}
```

→ // to remove later on...

→ // wait for tasks completion

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();
    sem_X_finished = xSemaphoreCreateCounting(10, 0);
    sem_Z_finished = xSemaphoreCreateCounting(10, 0);
    mbx_X = xQueueCreate(10, sizeof(int));
    mbx_Z = xQueueCreate(10, sizeof(int));
    xTaskCreate(goto_X_task, "GOTO X Task", 100, NULL, 0, NULL);
    xTaskCreate(goto_Z_task, "GOTO Z Task", 100, NULL, 0, NULL);
    xTaskCreate(goto_XZ_task, "GOTO XZ Task", 100, NULL, 0, NULL);
}
```

☐ Run the project...



Passing Parameters in Task Creation

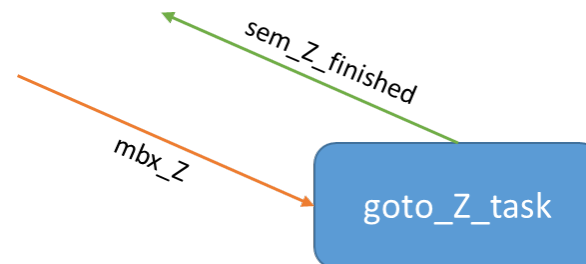
- ❑ As known, it is not a **good practice** to use global variables, especially in a real time context!!!

```
//Semaphores and Mailboxes declaration  
xSemaphoreHandle sem_X_finished;  
xSemaphoreHandle sem_Z_finished;  
  
xQueueHandle mbx_X; //for goto_X  
xQueueHandle mbx_Z; //for goto_Z
```



- Let's consider the creation of goto_Z_task. This task uses a mailbox (mbx_Z) and a semaphore (sem_Z_finished).
- It is then necessary to pass both task mechanisms in the **creation of the task**.
- For that, we suggest to create a structure with both mechanism handlers:

```
typedef struct taskGotoZParams {  
    xQueueHandle mbx_Z;  
    xSemaphoreHandle sem_Z_finished;  
} TaskGotoZParams;
```



The way you organize your code/structures is up to you! This is just an example.

Passing Parameters in Task Creation

❑ In myDaemonTaskStartupHook function:

- 1 – create the corresponding mechanism handlers;
- 2 – dynamically allocate memory to the structure (using the pvPortMalloc freeRTOS functionality) and assign value to its attributes;
- 3 – pass as argument the dynamically created pointer to the structure.



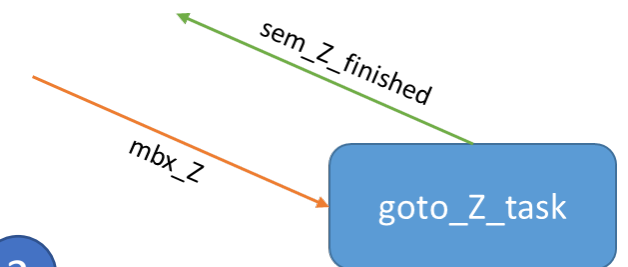
Attention! It is not recommended to pass the address of a stack variable!

```
void myDaemonTaskStartupHook(void) {  
  
    inicializarPortos();  
  
    xQueueHandle mbx_Z = xQueueCreate(10, sizeof(int));  
    xSemaphoreHandle sem_Z_finished = xSemaphoreCreateCounting(10, 0);  
    /* ... others ... */  
  
    TaskGotoZParams *my_taskgotoZ_Params = (TaskGotoZParams*)pvPortMalloc(sizeof(TaskGotoZParams));  
    my_taskgotoZ_Params->mbx_Z = mbx_Z;  
    my_taskgotoZ_Params->sem_Z_finished = sem_Z_finished;  
    /*... others ...*/  
  
    xTaskCreate(goto_Z_task, "GOTO Z Task", 100, my_taskgotoZ_Params, 0, NULL);  
    /* ... others ... */  
}
```

1

2

3



Passing Parameters in Task Creation



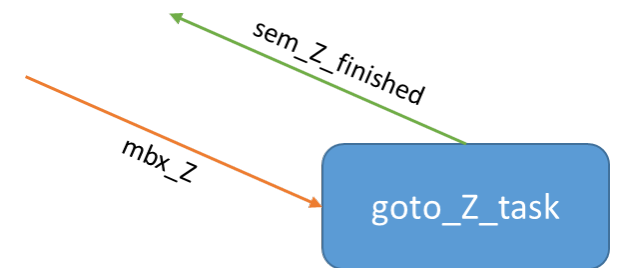
- In `goto_Z_task`:
 - receive the pointer to the structure and the corresponding fields

```
void goto_Z_task(void* pvParameters) {  
    int z;
```

```
    TaskGotoZParams* my_params = (TaskGotoZParams*)pvParameters;  
    xQueueHandle mbx_Z = my_params->mbx_Z;  
    xSemaphoreHandle sem_Z_finished = my_params->sem_Z_finished;
```

```
    while (TRUE) {  
        xQueueReceive(mbx_Z, &z, portMAX_DELAY);  
        gotoZ(z);  
        xSemaphoreGive(sem_Z_finished);  
    }
```

```
}
```





KEEP UP THE
GOOD
WORK!